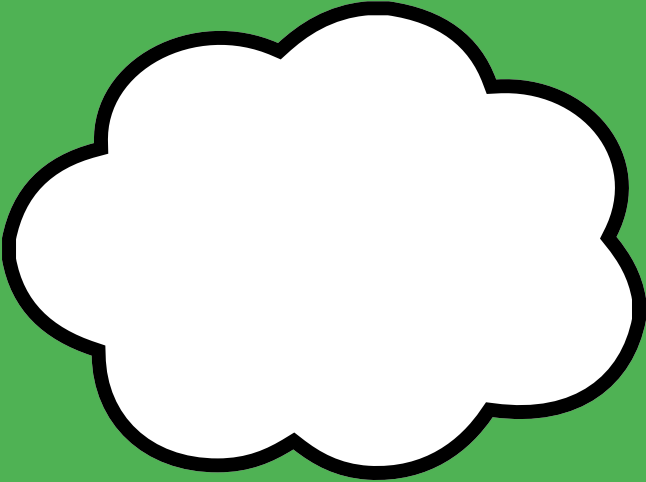
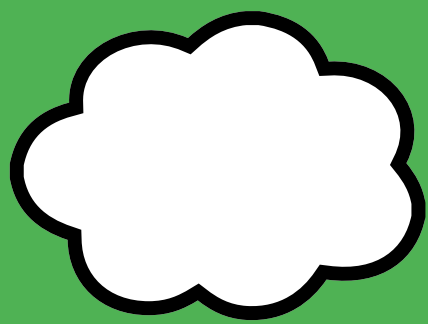


SQL PROJECT ON PIZZA SALES

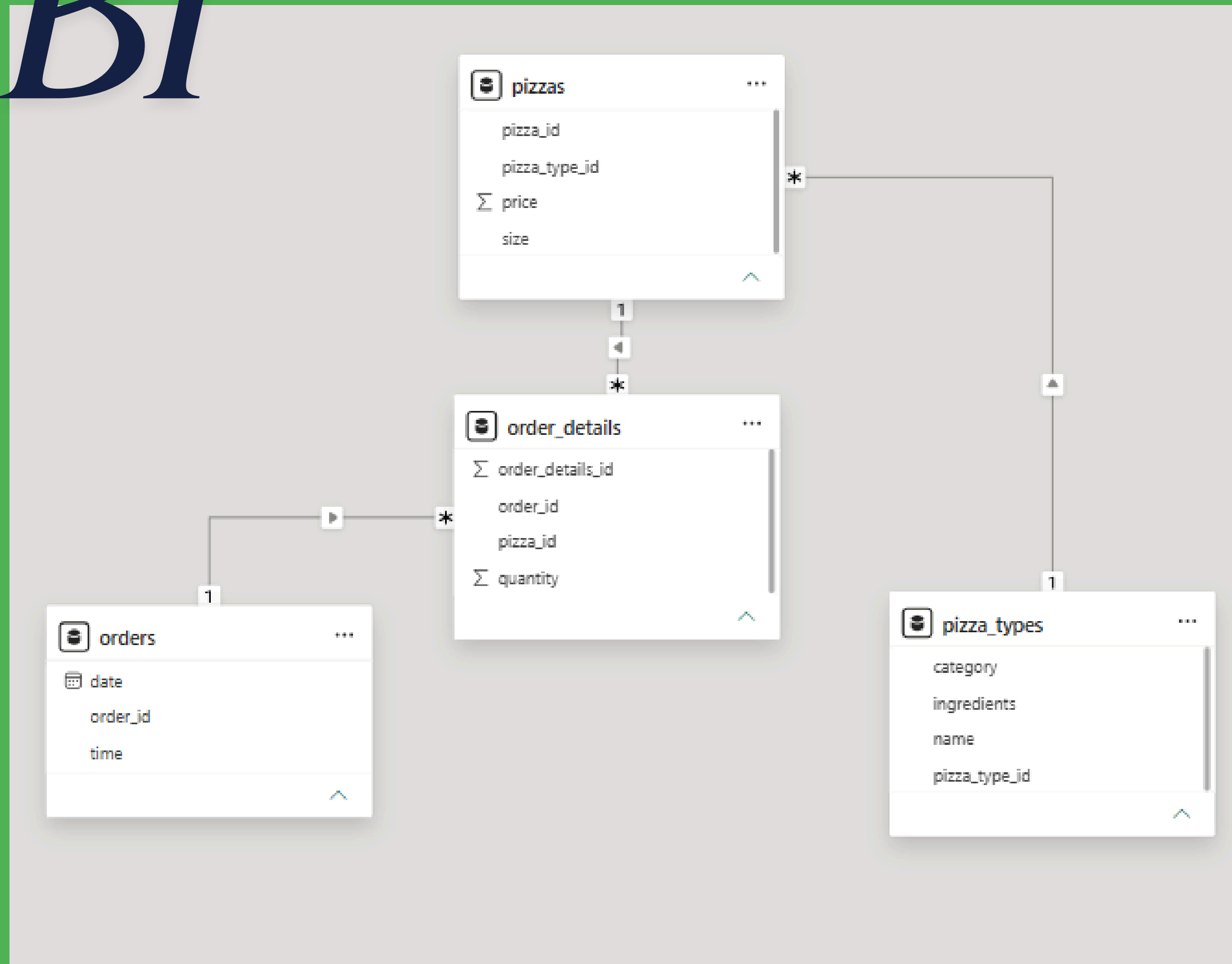
**22nd JUNE 2026
USING MYSQL**



"HEY EVERYONE! I'M CHRIS. IN THIS PROJECT, I USED MYSQL TO ANALYZE A PIZZA RESTAURANT'S SALES DATA – WRITING QUERIES TO ANSWER REAL BUSINESS QUESTIONS ABOUT REVENUE, ORDER VOLUME, AND CUSTOMER ORDERING PATTERNS."



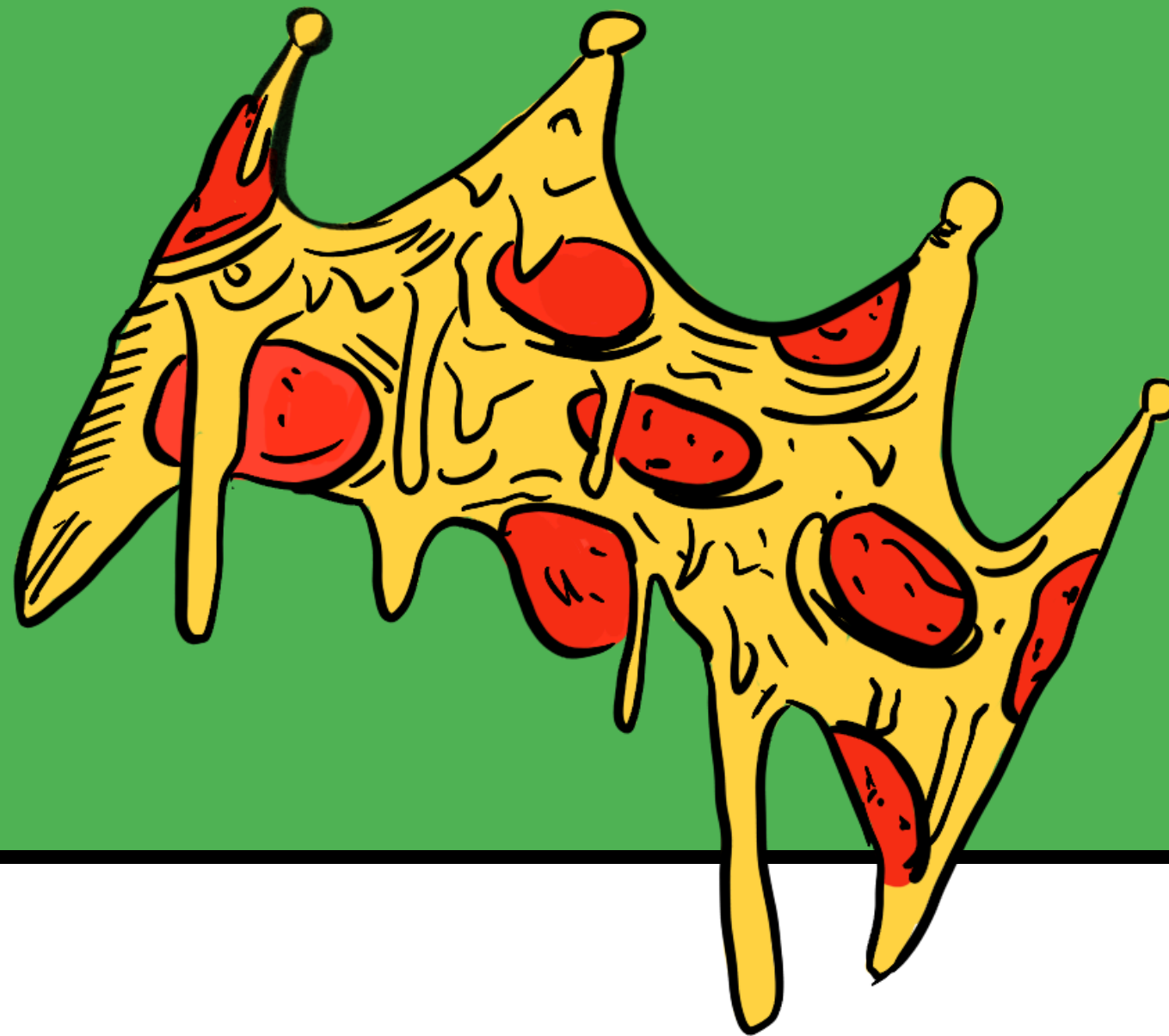
MODEL VIEW USING POWER BI



Retrieve the total number of orders placed

```
1 Retrieve the total number of orders placed.  
2 • select count(order_id) as total_orders from orders;
```

Result Grid	
	total_orders
▶	21350



Calculate the total revenue generated from pizza sales.

```
1 • SELECT
2     ROUND(SUM(order_details.quantity * pizzas.price),
3           2) AS total_sales
4 FROM
5     order_details
6     JOIN
7     pizzas ON order_details.pizza_id = pizzas.pizza_id;
```

Result Grid	
	total_sales
▶	817860.05



Identify the highest-priced pizza.

```
1 • SELECT
2     pizza_types.name, pizzas.price
3 FROM
4     pizza_types
5     JOIN
6     pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
7 ORDER BY pizzas.price DESC
8 LIMIT 1;
```

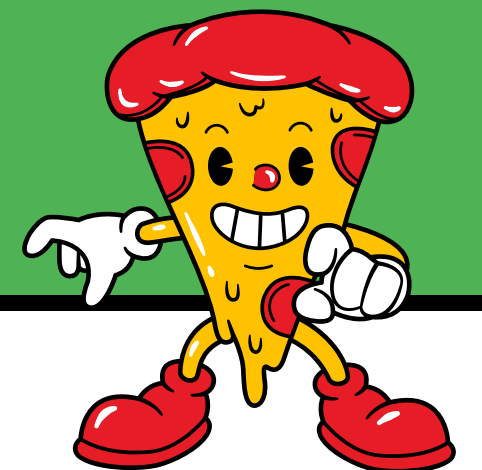
Result Grid			Filter Rows
	name	price	
▶	The Greek Pizza	35.95	



List the top 5 most ordered pizza types along with their quantities.

```
2 • SELECT
3     pizza_types.name, SUM(order_details.quantity) AS quantity
4 FROM
5     pizza_types
6     JOIN
7     pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
8     JOIN
9     order_details ON order_details.pizza_id = pizzas.pizza_id
10 GROUP BY pizza_types.name
11 ORDER BY quantity DESC
12 LIMIT 5;
```

Result Grid			Filter Rows:
	name	quantity	
▶	The Classic Deluxe Pizza	2453	
	The Barbecue Chicken Pizza	2432	
	The Hawaiian Pizza	2422	
	The Pepperoni Pizza	2418	
	The Thai Chicken Pizza	2371	



Join the necessary tables to find the total quantity of each pizza category ordered.

```
2 • SELECT
3     pizza_types.category,
4     SUM(order_details.quantity) AS quantity
5 FROM
6     pizza_types
7     JOIN
8     pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
9     JOIN
10    order_details ON order_details.pizza_id = pizzas.pizza_id
11 GROUP BY pizza_types.category
12 ORDER BY quantity DESC;
```

Result Grid			Filter
	category	quantity	
▶	Classic	14888	
	Supreme	11987	
	Veggie	11649	
	Chicken	11050	



Determine the distribution of orders by hour of the day.

```
2 • SELECT
3     HOUR(order_time) AS hour, COUNT(order_id) AS order_id
4 FROM
5     orders
6 GROUP BY HOUR(order_time);
```

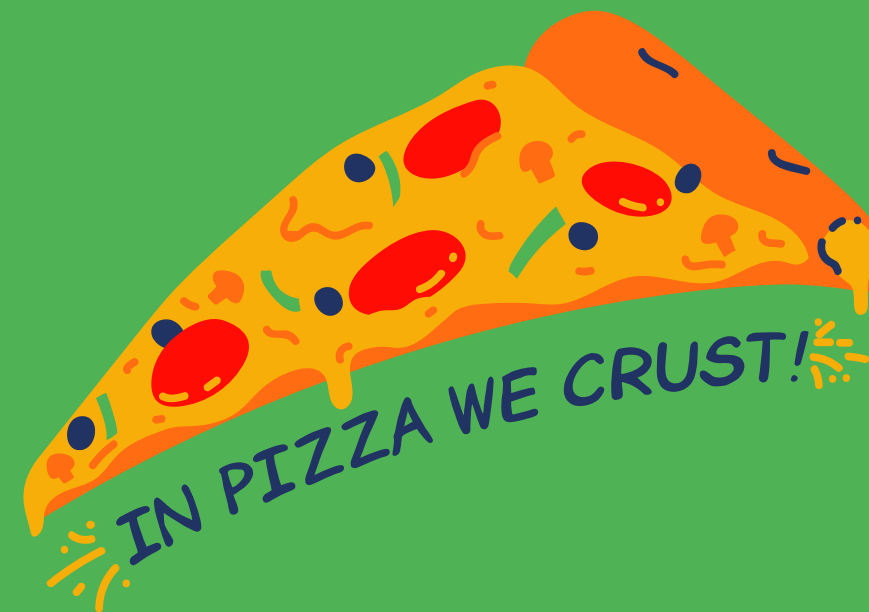
Result Grid				Filter
	hour	order_id		
▶	11	1231		
	12	2520		
	13	2455		
	14	1472		
	15	1468		
	16	1920		
	17	2336		
	18	2399		
	19	2009		
	20	1642		
	21	1198		
	22	663		
	23	28		
	10	8		
	9	1		



Join relevant tables to find the category-wise distribution of pizzas.

```
2 ● SELECT
3     category, COUNT(name)
4 FROM
5     pizza_types
6 GROUP BY category;
```

Result Grid			Filter Rows
	category	COUNT(name)	
▶	Chicken	6	
	Classic	8	
	Supreme	9	
	Veggie	9	



Group the orders by date and calculate the average number of pizzas ordered per day.



```
2
3  SELECT
4  ROUND(AVG(quantity), 0) as average_pizza_ordered_per_day
5  FROM
6  (SELECT
7    orders.order_date, SUM(order_details.quantity) AS quantity
8  FROM
9    orders
10 JOIN order_details ON orders.order_id = order_details.order_id
11 GROUP BY orders.order_date) AS order_quantity;
```

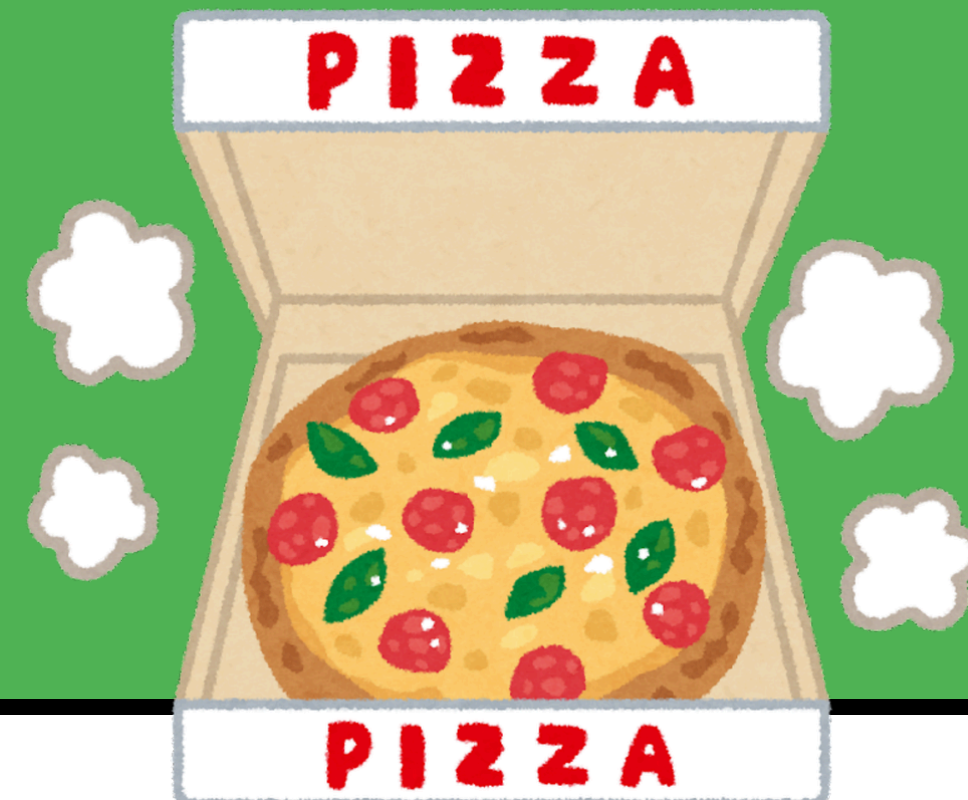
Result Grid		Filter Rows:
	average_pizza_ordered_per_day	
▶	138	



Determine the top 3 most ordered pizza types based on revenue.

```
2  SELECT
3      pizza_types.name,
4      SUM(order_details.quantity * pizzas.price) AS total_revenue
5  FROM
6      pizzas
7      JOIN
8      pizza_types ON pizzas.pizza_type_id = pizza_types.pizza_type_id
9      JOIN
10     order_details ON order_details.pizza_id = pizzas.pizza_id
11  GROUP BY pizza_types.name
12  ORDER BY total_revenue DESC
13  LIMIT 3;
```

Result Grid			Filter Rows:
	name	total_revenue	
▶	The Thai Chicken Pizza	43434.25	
	The Barbecue Chicken Pizza	42768	
	The California Chicken Pizza	41409.5	



Calculate the percentage contribution of each pizza type to total revenue.

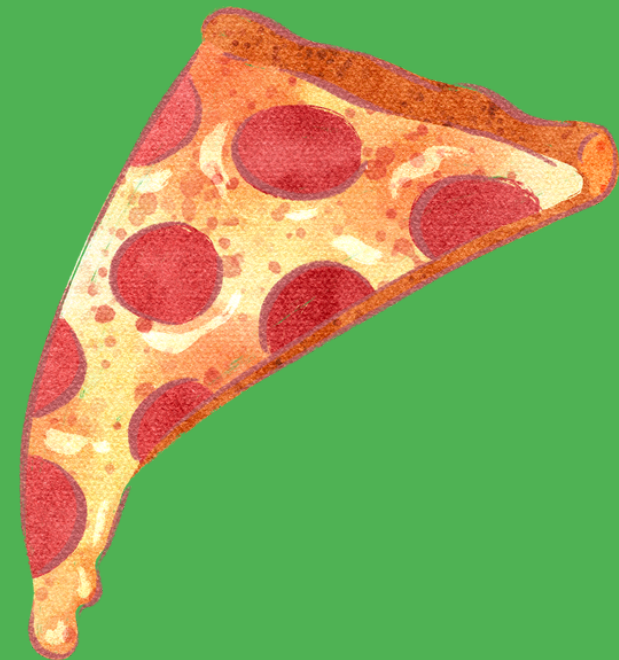
```
2  SELECT
3      pizza_types.category,
4      ROUND((SUM(order_details.quantity * pizzas.price) / (SELECT
5          ROUND(SUM(order_details.quantity * pizzas.price),
6              2) AS total_sales
7          FROM
8              order_details
9              JOIN
10                 pizzas ON order_details.pizza_id = pizzas.pizza_id)) * 100,
11          2) AS revenue_percent
12  FROM
13      pizza_types
14      JOIN
15      pizzas ON pizza_types.pizza_type_id = pizzas.pizza_type_id
16      JOIN
17      order_details ON order_details.pizza_id = pizzas.pizza_id
18  GROUP BY pizza_types.category
19  ORDER BY revenue_percent DESC;
```

Result Grid			Filter Rows:
	category	revenue_percent	
▶	Classic	26.91	
	Supreme	25.46	
	Chicken	23.96	
	Veggie	23.68	

Analyze the cumulative revenue generated over time.

```
2 • SELECT
3     order_date,
4     SUM(revenue) OVER (ORDER BY order_date) AS cum_revenue
5 FROM (
6     SELECT
7         orders.order_date,
8         SUM(order_details.quantity * pizzas.price) AS revenue
9     FROM order_details
10    JOIN pizzas
11     ON order_details.pizza_id = pizzas.pizza_id
12    JOIN orders
13     ON orders.order_id = order_details.order_id
14    GROUP BY orders.order_date
15 ) AS sales;
```

Result Grid			Filter Rows:
	order_date	cum_revenue	
▶	2015-01-01	2713.8500000000004	
	2015-01-02	5445.75	
	2015-01-03	8108.15	
	2015-01-04	9863.6	
	2015-01-05	11929.55	
	2015-01-06	14358.5	
	2015-01-07	16560.7	
	2015-01-08	19399.05	
	2015-01-09	21526.4	
	2015-01-10	23990.350000000002	
	2015-01-11	25862.65	
	2015-01-12	27781.7	
	2015-01-13	29831.300000000003	
	2015-01-14	32358.700000000004	
	2015-01-15	34343.500000000001	
	2015-01-16	36937.650000000001	
	2015-01-17	39001.750000000001	
	2015-01-18	40978.600000000006	
	2015-01-19	43365.750000000001	
	2015-01-20	45763.650000000001	



Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SELECT *
FROM (
    SELECT category,
           name,
           revenues,
           RANK() OVER (
               PARTITION BY category
               ORDER BY revenues DESC
           ) AS rn
    FROM (
        SELECT pizza_types.category,
               pizza_types.name,
               SUM(order_details.quantity * pizzas.price) AS revenues
        FROM pizza_types
        JOIN pizzas
        ON pizza_types.pizza_type_id = pizzas.pizza_type_id
        JOIN order_details
        ON order_details.pizza_id = pizzas.pizza_id
        GROUP BY pizza_types.category, pizza_types.name
    ) a
    ) b
WHERE rn <= 3;
```

	category	name	revenues	rn
▶	Chicken	The Thai Chicken Pizza	43434.25	1
	Chicken	The Barbecue Chicken Pizza	42768	2
	Chicken	The California Chicken Pizza	41409.5	3
	Classic	The Classic Deluxe Pizza	38180.5	1
	Classic	The Hawaiian Pizza	32273.25	2
	Classic	The Pepperoni Pizza	30161.75	3
	Supreme	The Spicy Italian Pizza	34831.25	1
	Supreme	The Italian Supreme Pizza	33476.75	2
	Supreme	The Sicilian Pizza	30940.5	3
	Veggie	The Four Cheese Pizza	32265.700000000065	1
	Veggie	The Mexicana Pizza	26780.75	2
	Veggie	The Five Cheese Pizza	26066.5	3



Thanks for reading! If you found this useful, drop your thoughts in the comments — let's connect.

— CHRIS



<https://github.com/chrislincoln-ctrl>



<https://ciyx4rxljm6z.trickle.host/>



<https://www.linkedin.com/in/chris-tejas-lincoln-53553933b/>